UrbanTrajectoryDB：真实轨迹数据的存储、查询与压缩实验

实验日期：2026-10-13。本文记录GeoLife数据导入、PostGIS索引对照和C++轨迹压缩的实现方法与测量结果。

1. 问题与目标

城市轨迹由大量带时间的空间点组成。普通逐行读取能看到坐标，但如何可靠导入、按时空条件检索、解释索引是否有效，以及在误差约束下减少几何点数，是本项目关注的问题。

实验问题：

1. 文件级事务和来源校验能否防止坏文件部分入库、重复导入？

2. GiST空间索引、B-tree时间索引对不同范围查询有什么影响？

3. Douglas–Peucker的误差阈值如何影响保留点数？

2. 数据与实现

实验数据来自微软发布的GeoLife 1.3原始压缩包。按照路径排序选取前1,000个PLT文件，涉及5个用户，共1,316,180点。此选择可复现但不是随机样本，不能代表全数据集或总体人群。

数据时间范围为2008-10-23 02:53:04至2009-07-05 07:45:15（UTC）。官方用户指南第4.1节明确日期时间采用GMT；导入器据此将解析器的无时区对象解释为UTC。海拔继续以英尺保存，官方缺失标记-777入库为NULL。源文件逐个保存SHA-256与点数。

首次导入结果：1,000成功、0失败；再次导入：1,000跳过、0新增。集成测试使用人工构造的固定样例验证坏文件回滚、内容变更拒绝、查询和来源记录；这些样例不参与下面的真实数据性能结论。

结构：Python解析与校验 → psycopg COPY/事务 → PostgreSQL/PostGIS → 参数化查询与EXPLAIN → C++压缩。

数据库点几何为geometry(Point,4326)，按经度在前、纬度在后构造。查询支持用户、时间范围、矩形空间窗口。时间和窗口端点包含，跨日期变更线窗口暂不支持。CLI结果限量，不会一次把百万行输出到终端。

3. 索引实验方法

实测环境：Windows主机上的Docker Desktop，PostgreSQL 17.11 / PostGIS 3.5.7，Python 3.14.7。详尽数据库版本与优化器设置见原始plans.json；只报告本机结果，不外推生产服务性能。

每轮创建独立实验schema，复制真实源表为无索引堆表，源表不变。先测无索引基线，再创建GiST(geom)和B-tree(recorded_at)，两个阶段均ANALYZE。最后删除本轮独立schema。

空间、时间、组合三类查询分别取数据包围范围/时间跨度的0.01、0.1、1.0中心窗口，形成9种工作负载。名义比例不是实际返回比例，实际选择性记录在CSV。每种每阶段预热1次、测量5次，取服务器Execution Time中位数；这是热缓存实验，不含客户端传输、并发压力或索引构建时间。

使用EXPLAIN(ANALYZE,BUFFERS,FORMAT JSON)，不关闭顺序扫描、不强制优化器走索引。每轮校验实际返回行数和独立COUNT，并验证建索引前后行数一致。共18条中位数记录、90份测量计划。

4. 索引实测结果

查询	返回点数	无索引 ms	有索引 ms	比值（无/有）	有索引时计划
spatial_0.01	2380	98.567	0.346	284.88	Bitmap Heap Scan;Bitmap Index Scan

查询	返回点数	无索引 ms	有索引 ms	比值（无/有）	有索引时计划
time_0.01	14537	61.239	2.129	28.76	Bitmap Heap Scan;Bitmap Index Scan
spatial_0.1	6612	100.237	1.399	71.65	Bitmap Heap Scan;Bitmap Index Scan
time_0.1	143860	86.941	18.103	4.80	Bitmap Heap Scan;Bitmap Index Scan
spatial_1	1316180	284.911	283.825	1.00	Seq Scan
time_1	1316180	162.363	165.347	0.98	Seq Scan
combined_1	1316180	294.347	314.928	0.93	Seq Scan

如何解释

- 低实际选择性的窗口在本机走Bitmap Index/Heap Scan，明显降低扫描开销。

- 全范围查询仍自然选择Seq Scan。时间全范围和组合全范围的有索引阶段甚至略慢，不应解读成“有索引必然更快”；小差异也可能来自缓存与调度波动。

- combined_0.01与combined_0.1返回0行。原始结果完整保留，但不能用这些空结果宣称真实业务组合查询获得巨大提速。后续应增加按真实点分布选择的非空查询窗口。

- 当前基线和索引阶段顺序固定，虽然每种均预热，仍可能有顺序效应；更严谨实验可做交错多轮、置信区间及不同缓存条件。

5. C++几何压缩实验

选用已导入的第一条真实轨迹，908点。C++17标准库实现迭代Douglas–Peucker：保留端点，按点到有限线段的距离递归思想分割，使用显式栈避免递归栈风险。

为了得到可理解的近似米尺度，以第一点为原点/参考纬度做局部等距柱状投影；不是全球椭球测地距离保证。每次输出前独立审计所有原始顶点到返回折线的最小投影距离，确认最大值不超过ε。

ε（投影米）	原始点	保留点	移除比例	实测最大偏差（投影米）
1	908	536	40.97%	0.9965
5	908	198	78.19%	4.9858
10	908	111	87.78%	9.8673
20	908	69	92.40%	19.4686
50	908	34	96.26%	46.5965

移除比例是点数比例，不是文件字节压缩率。这里仅测试一条局部轨迹，不能代表所有轨迹。输出保留原始顺序的经纬度，但当前C++CSV没有时间或海拔字段，因此它是几何模块，不是无损恢复、速度保真、时间同步误差或完整数据库压缩方案。测地误差、长途轨迹、极区、跨日期变更线仍需改进。DP最坏$O(n^2)$，审计$O(n \times 保留点数)$，暂无算法运行时间对照实验。

6. 可复现性与证据

最终本机验证：启用真实PostGIS集成测试后64 passed，Ruff对src/tests/scripts检查通过；数据库容器healthy，仅监听127.0.0.1。测试通过不代表所有未来输入都正确，未覆盖范围见限制。

- docs/evidence/index-medians.csv：18条真实记录。

- docs/evidence/index-plans.json：90份原始计划、参数、版本和来源SHA。

- docs/evidence/showcase-summary.json：真实导入/重复导入与压缩统计。

- scripts/run_showcase.py：一条命令重新导入（可跳过已导入文件）、查询、跑索引实验、编译并跑压缩。

- docs/database-usage.md、docs/compression.md：详细接口与限制。

原始数据及压缩轨迹CSV仅保留在git-ignored本地目录；不随代码仓库分发。数据使用条件和引用应以下载包的User Guide及微软官方页面为准。

7. 结论与下一步

数据解析、事务导入、时空查询和索引实验已串联运行；在本机低选择性查询中索引有效，而全范围查询没有稳定收益；压缩阈值增加会显著减少单条轨迹的几何点数，并在所用投影误差定义下通过审计。

后续可扩展：非空且代表性的工作负载、全量/随机样本、多轮统计、联合或分区索引、保持时间语义的轨迹压缩、更准确的空间度量。暂未完成AI4DB、分布式数据库、异常速度过滤或生产级服务。

参考资料

1. Microsoft Research，GeoLife GPS Trajectories 1.3，随数据包User Guide-1.3.pdf（第4.1节数据格式，第7节使用条件）。

2. PostgreSQL官方文档：EXPLAIN、索引与COPY。

3. PostGIS官方文档：ST_MakePoint、ST_MakeEnvelope与GiST。

4. Douglas, D. H. & Peucker, T. K. (1973), Algorithms for the reduction of the number of points required to represent a digitized line or its caricature. 本项目的几何压缩模块基于该算法实现。